Takum Arithmetic for the RISC-V Vector Extension: Encoding, Functional Simulation, and Evaluation

Grupo do Projeto Integrador AOC – UFMG
Departamento de Engenharia Eletronica
Belo Horizonte, MG, Brasil

***Abstract*—Modern vector ISAs accumulate many low-precision floating-point formats (bfloat16, FP8 variants) that inherit IEEE 754 complexities and multiply instruction-set surface. Takum arithmetic has been proposed as a tapered-precision alternative with a bounded dynamic range and a common 12-bit decoder. This work replicates, on the RISC-V Vector extension (RVV), the methodology of the AVX10.2 case study: we specify a Zvtakum extension (`vtadd.vv`, `vtsub.vv`, `vtmul.vv`, `vtdot.vv`, `vncvt.t.t`) for T8/T16/T32, implement a reference codec and ALU on top of libtakum (both linear and logarithmic variants), build a functional RVV simulator with takum support and instruction/CSR counters, and evaluate three axes: representational viability on 100 SuiteSparse matrices, arithmetic closure for T8/T16, and sparse matrix-vector multiplication. Takum T32 outperforms IEEE binary32 in the matrix-conversion benchmark, closure percentages match the literature within sampling noise, and the proposed instruction set replaces the 96 floating-point/conversion instructions of RVV's current format zoo with eight takum operations.**

***Index Terms*—RISC-V, RVV, takum arithmetic, tapered precision, Zvtakum, SIMD, ISA design**

I. Introduction

Vector instruction sets increasingly support a zoo of low-precision formats – bfloat16, FP8 E4M3/E5M2, FP16 – aiming at machine-learning and low-precision HPC workloads [1]–[3]. Each format adds conversions, exception corners and decoder area; in RVV, FP8 is still absent while bfloat16 required two dedicated extensions [4].

Takum [5] is a tapered-precision format with a bounded dynamic range covering the whole range at 12 bits, implying that all element widths share the same decoder – a property that simplifies hardware and instruction-set design. The AVX10.2 case study [6] showed, for Intel AVX10.2, that replacing the multiple arithmetic formats by takum (T8/T16/T32/T64) both simplifies the ISA and either matches or improves accuracy; its conclusion explicitly suggests studying RISC-V and ARM vector extensions.

This paper performs that study for RVV. The contributions are:

- a Zvtakum specification for T8/T16/T32 (Sections III), with provisional encodings derived from the single free OP-V `funct6` slot;
- a reference implementation and a functional RVV simulator with instruction/CSR counters and automated tests (exhaustive for T8);
- three evaluations: representational viability of 15 formats on 100 SuiteSparse matrices, arithmetic-closure reproduction of [5], and SpMV kernels in the simulator;
- an RVV–Takum restructuring analysis mirroring the AVX10.2 tables.

II. Background and Related Work

A. Takum

An n-bit takum is a string (S, D, R, C, F): sign S, direction D, three regime bits R, $r \in \{0, \ldots, 7\}$ characteristic bits C and $p = n - r - 5$ fraction bits. The *linear* variant (Definition 8 of [5]) represents $v = [(1-3S)+f]\,2^e$ with $e = (-1)^S(c+S)$; the *logarithmic* variant (its standard) represents $v = (-1)^S\sqrt{e}^{\ell}$ with $\ell = (-1)^S(c+m)$. Bit strings shorter than 12 bits are zero-extended ("ghost bits"), which bounds the characteristic part to 11 bits and guarantees a common decoder. Negation is two's-complement negation of the bit string; NaR (Not a Real) is the pattern $100\ldots0$, ordered below all values; arithmetic is saturating. For $n = 8$ the effective dynamic range is reduced ($2^{\pm 239}$ linear, $e^{\pm 119.5}$ logarithmic) because the characteristic/fraction bits are partially ghosts.

B. Posits and IEEE derivatives

Posits [7] introduced tapered precision but suffer from unbounded exponent codes; bfloat16 [3] and OFP8 [2] trade range for tiny representations. The AVX10.2 study [6] benchmarked all of these on 1401 SuiteSparse matrices and restructured AVX10.2 into a takum-based ISA. Our work reuses the same methodology on RVV with an open implementation: codec and ALU over libtakum [8], simulator, and benchmarks, complementing the takum hardware codec [9] and the multi-format benchmark tool [10].

III. Methodology

A. Codec and ALU

The codec decodes fields through the 12-bit ghost expansion and computes values independently of the arithmetic library; conversions and operations delegate to libtakum v2.0.0 [8]. Two rounding modes are implemented: round-to-nearest-even (RNE, normative) and Algorithm 1 of [5] (bit-string guard rounding with saturation). On 200,000 random inputs per variant and width no divergence was observed, but exact ties were not sampled. Integer comparisons follow the two's-complement order, and NaR is detected by the `INT_MIN` patterns. The codec/ALU were validated exhaustively for T8 (all

TABLE I
PROPOSED ZVTAKUM INSTRUCTIONS (SEW SELECTS T8/T16/T32).

Mnemonic	Operation	Notes
`vtadd.vv`	$vd_i = vs2_i + vs1_i$	saturating, NaR propagates
`vtsub.vv`	$vd_i = vs2_i - vs1_i$	idem
`vtmul.vv`	$vd_i = vs2_i \times vs1_i$	idem
`vtdot.vv`	$vd_0 \mathrel{+}= \sum_i vs2_i \times vs1_i$	widening accumulator
`vncvt.t.t`	$T_{2n} \to T_n$	narrowing, RNE

256 bit patterns; all 256^2 operand pairs for add/sub/mul/div) and by sampled property tests for T16/T32: monotonicity, negation, logarithmic inversion, NaR propagation and saturation boundaries.

B. Zvtakum instructions

Table I lists the proposed instructions. Element width is selected by `vtype.SEW` (8/16/32). `vtdot.vv` accumulates into `vd[0]` with graded widening (T8→T16, T16→T32, T32→T64), matching the proposed `VDPP` family of [6]; the destination uses EEW $= 2 \cdot$ SEW, and the accumulator must be initialized by software. `vncvt.t.t` narrows $T_{2n} \to T_n$ like `vfncvt`. Encodings use the only free OP-V `funct6` (`0b111001`) with `funct3` as a sub-opcode and bit 25 zero – a provisional choice documented in the repository. Masking follows RVV: `v0` mask, `vta`/`vma` tail policy, no new CSRs, no FP exceptions, rounding fixed to RNE.

C. RVV restructuring

Using the machine-readable inventory of `riscv-opcodes` (2026-09), RVV 1.0 has 375 instructions, of which 92 are floating point (including 21 conversions and 6 FP reductions); bfloat16 adds 4 instructions and vector crypto 41. No FP8 exists in RVV. Applying the AVX10.2 guidelines (grouping, systematic naming B/U/S/T, format consolidation, generalization), the 96 FP/conversion instructions collapse into eight takum operations: the five above plus an integer conversion pair and a reserved division/square-root pair. Comparisons become integer comparisons; bf16 and FP16 use cases are subsumed by T16/T8.

D. Functional simulator and benchmarks

The simulator implements a RVV subset (vsetvl*, unit-stride loads/stores, `vfadd/vfmul` in binary32, `vfredusum`, `vrgather.vv`, moves) plus Zvtakum over a configurable VLEN, with instruction, CSR-write and active-element counters. Three benchmarks are used: *(i) representational viability*: 100 SuiteSparse matrices with `nnz`$\leq$50,000 are converted to 15 formats and back to a `long double` reference, accumulating the relative 2-norm error $\|r - a\|_2 / \|r\|_2$ (the article uses all 1401 matrices and binary128; we document the subset and use 80-bit extended precision, which is far more accurate than any tested format); *(ii) closure*: for T8 (exhaustive) and T16 (2,000,000 sampled pairs) within the posit range, the fraction of operations whose result is exactly representable, following Section 5.6 of [5]; *(iii)* *SpMV* $y = Ax$ with `vrgather`+`vtdot` (takum) and `vrgather`+`vfmul`+`vfredusum` (binary32), reference in `long double` over format-rounded inputs.

TABLE II
REPRESENTATIONAL VIABILITY (100 MATRICES; RELATIVE 2-NORM ERROR).

Format	finite	median	p90	overflow
FP8 E4M3	73	7.2e-3	2.7e-2	27
FP8 E5M2	82	2.4e-2	5.7e-2	18
takum8	100	5.7e-2	3.7e-1	0
takum8 (log)	100	5.4e-2	3.6e-1	0
posit8	100	2.5e-2	6.1e-1	0
FP16	82	5.8e-6	2.2e-4	18
bfloat16	100	9.5e-4	1.8e-3	0
takum16	100	1.5e-4	1.6e-3	0
takum16 (log)	100	2.2e-4	1.3e-3	0
posit16	100	8.4e-5	2.8e-3	0
binary32	100	1.0e-8	3.0e-8	0
takum32	100	1.1e-9	2.1e-8	0
takum32 (log)	100	3.6e-9	1.9e-8	0
posit32	100	1.0e-9	3.1e-8	0

IV. RESULTS AND DISCUSSION

A. Representational viability

Table II summarizes the 100 matrices (median and 90th percentile of the relative 2-norm error; overflow counts where the format cannot represent matrix entries). At 8 bits, IEEE FP8 overflows on 18–27% of matrices, while takum saturates and keeps the error bounded for 97–98% of matrices; at 16 bits takum beats bfloat16 and is comparable to posit16; at 32 bits takum32 has the lowest median error (1.1e-9), better than binary32 (1.0e-8), reproducing the article's trend. The logarithmic and linear variants are within a factor of three of each other.

B. Arithmetic closure

Table III compares our logarithmic-takum closure with the published values [5]. Addition is never exact (0.00% in both), multiplication lands within sampling error (41.2% vs. 40.3% at T8; 36.7% vs. 36.6% at T16), inversion is exactly representable by construction (100%), and square roots match (60.5% vs. 58.8%; 84.2% vs. 84.1%). The residual differences are explained by sampling, the 80-bit reference and the two rounding modes.

C. SpMV in the simulator

Table IV reports the relative 2-norm error of y and executed instructions. Takum T32 is about 15$\times$ more accurate than binary32 in the random case (2.8e-9/3.5e-9 vs. 5.1e-8), T16 stays near 1e-4 and T8 near 3–6e-2, tracking the representation results. The takum kernel needs $\sim$20% more instructions than the binary32 kernel (193 vs. 161 on the random matrix) because the reduction is followed by a narrowing conversion and a scalar store, but it replaces the whole low-precision instruction surface; CSR traffic is one `vsetvli` per strip in both kernels.

TABLE III
CLOSURE OF LOGARITHMIC TAKUM (% EXACT); VALUES IN PARENTHESES ARE FROM [5].

	T8		T16	
Op	ours	ref	ours	ref
add	0.00	0.0	0.00	0.0
sub	2.44	1.9	0.01	0.0
mul	41.17	40.3	36.74	36.6
div	41.17	40.3	36.75	36.7
inv	100.0	100.0	100.0	100.0
sqrt	60.49	58.8	84.20	84.1
square	60.49	60.0	100.0	100.0

TABLE IV
SPMV: RELATIVE 2-NORM ERROR OF y (REFERENCE: LONG DOUBLE) AND EXECUTED INSTRUCTIONS.

Matrix	Kernel	rel. error	insns
lap2d	binary32	0.0	126
lap2d	takum T8	2.8e-2	151
lap2d	takum T16	1.3e-4	151
lap2d	takum T32	2.3e-9	151
rand	binary32	5.1e-8	161
rand	takum T8	5.8e-2	193
rand	takum T16	1.3e-4	193
rand	takum T32	3.5e-9	193

D. ISA and CSR complexity

The current RVV format zoo (FP16 via `Zvfh`, bfloat16 via `Zvfbfmin`/`Zvfbfwma`, binary32/64) requires 96 FP/conversion instructions and 21 conversion mnemonics. Zvtakum covers the same low-precision ground with five implemented (eight proposed) instructions, one rounding mode and no new CSRs; comparisons reuse integer ordering. This mirrors the AVX10.2 result and supports the article's claim that a single taper-precision base can replace the proliferation of derived formats.

E. Limitations

The benchmark uses 100 matrices (not 1401) and an 80-bit reference (not binary128); the simulator supports LMUL 1–8 but no fractional LMUL, only binary32 among IEEE operations, and small SpMV matrices because `vrgather` indexes are bounded by VLMAX. Encodings are provisional (the OP-V space would have to be negotiated with the RISC-V technical committees). The logarithmic variant follows libtakum's RNE rounding; Algorithm 1 differed only at unsampled exact ties.

V. CONCLUSION

We replicated the AVX10.2-to-takum methodology on RISC-V's vector extension: a Zvtakum specification for T8/T16/T32, a validated codec/ALU (both takum variants), a functional RVV simulator with counters, and three evaluations. Takum T32 matches or beats binary32 in representational error, closure percentages reproduce the literature, and the proposed instruction set shrinks RVV's low-precision FP surface from 96 to eight operations without new CSRs or exception modes. Future work: full 1401-matrix run, hardware codec/decoder RTL and FPGA comparison, vector `.vf` forms and mixed-precision kernels.

REPRODUCIBILITY

All code, specifications, benchmarks and raw results are available in the project repository (private): `riscv-takum` – codec/ALU, Zvtakum specification, encodings, simulator, tests (`make test`) and benchmarks (`make bench`).

REFERENCES

[1] "IEEE standard for floating-point arithmetic," IEEE, Tech. Rep. IEEE Std 754-2019, 2019.

[2] P. Micikevicius, S. Oberman, P. Dubey *et al.*, "OCP 8-bit floating point specification (OFP8)," Open Compute Project, 2023.

[3] S. Wang and P. Kanwar, "BFloat16: The secret to high performance on cloud TPUs," 2019, google Cloud Blog.

[4] RISC-V International, "The RISC-V vector extension, version 1.0," https://github.com/riscv/riscv-v-spec, 2021.

[5] L. Hunhold, "Beating posits at their own game: Takum arithmetic," in *Next Generation Arithmetic (CoNGA 2024), LNCS vol. 14666*. Springer Nature Switzerland, 2024, arXiv:2404.18603.

[6] ——, "Streamlining SIMD ISA extensions with takum arithmetic: A case study on intel AVX10.2," in *2025 14th International Conference on Modern Circuits and Systems Technologies (MOCAST)*, 2025, pp. 1–6, arXiv:2503.14067.

[7] J. L. Gustafson and I. Yonemoto, "Beating floating point at its own game: Posit arithmetic," *Supercomputing Frontiers and Innovations*, vol. 4, no. 2, pp. 71–86, 2017.

[8] L. Hunhold, "libtakum: Takum arithmetic C99 reference implementation, v2.0.0," https://github.com/takum-arithmetic/libtakum, 2025.

[9] ——, "Design and implementation of a takum arithmetic hardware codec in VHDL," 2024, arXiv:2408.10594.

[10] L. Hunhold and J. Quinlan, "MuFoLAB – multi-format linear algebra benchmarks," 2025.